

Rapport Complet du Projet

Framework principal : Flutter 3.x
Backend : Firebase (Firestore, Auth, Storage)
Gestion d'état : Riverpod 2.5.0
Plateformes : Android, iOS, Web, Windows, Linux, macOS
État du projet : En développement actif
Date de création : 2024
Dernière mise à jour : 26 Décembre 2025

Auteur : ASAAD AZAROUAL

Table des matières

1	Résumé Exécutif	4
2	Description du Projet	6
2.1	Vision Générale	6
2.2	Public Cible	6
2.3	Proposition de Valeur	6
2.4	État Actuel du Projet	6
3	Objectifs et Périmètre	7
3.1	Objectifs Principaux	7
3.2	Périmètre	8
3.3	KPIs	8
4	Architecture Générale	9
4.1	Architecture Globale	9
4.2	Patterns Architecturaux	10
4.3	Flux de Données	10
4.4	Principes de Conception	10
5	Technologies Utilisées	11
5.1	Frontend	11
5.2	Backend & Services	11
5.3	Dépendances clés (extrait)	11
6	Structure du Projet	13
6.1	Hierarchie des dossiers	13
6.2	Arborescence lib/src	13
7	Fonctionnalités Principales	14
7.1	Gestion des recettes (héritage)	14
7.2	Authentification & profil	14
7.3	Fonctionnalités communautaires	14

7.4	Notifications	14
7.5	Gestion des données & sécurité	14
8	Modèles de Données	16
8.1	Modèle Recette	16
8.2	Modèle Utilisateur	16
8.3	Modèle Match	17
9	Services et Providers	18
9.1	Exemple Provider (StateNotifier)	18
10	Intégration Firebase	19
10.1	Initialisation Firebase	19
10.2	Exemples de requêtes Firestore	19
10.3	Règles de sécurité Firestore (extrait)	19
11	Écrans et Navigation	21
12	Configuration et Installation	22
12.1	Prérequis	22
12.2	Installation	22
12.3	Firebase (résumé)	22
13	Détails Techniques	23
13.1	Sérialisation JSON (json_serializable)	23
14	Gestion de l'État (Riverpod)	24
15	Plateformes Supportées	25
15.1	Mobile	25
15.2	Desktop	25
15.3	Web	25
16	Gestion des Dépendances	26
16.1	Extrait pubspec.yaml	26
16.2	Commandes utiles	26
17	Tests et Validation	27
18	Défis et Solutions	28
18.1	Synchronisation en temps réel	28
18.2	Sécurité	28
18.3	Performances	28

18.4 Multi-plateforme	28
19 Perspectives Futures	29
19.1 Court terme (1–3 mois)	29
19.2 Moyen terme (3–6 mois)	29
19.3 Long terme (6+ mois)	29
20 Conclusion	30

Chapitre 1

Résumé Exécutif

MatchUp Football App est une application mobile multi-plateforme développée avec **Flutter** et **Firebase**. Elle permet aux amateurs de football de :

- Organiser et rejoindre des matchs de football locaux
- Gérer les participants et les positions sur le terrain
- Communiquer en temps réel via des messages et des annonces
- Découvrir les stades disponibles dans leur région
- Suivre les matchs et s'inscrire facilement

Informations clés du projet

- Nom du projet : MatchUp Football App (anciennement Flutter Recipes App)
- Framework : Flutter 3.x
- Langages : Dart, Kotlin (Android), Swift (iOS), C++ (Desktop)
- Backend : Firebase (Firestore, Authentication, Storage)
- Gestion d'état : Riverpod 2.5.0
- Plateformes cibles : iOS, Android, Web, Windows, Linux, macOS
- État : En développement actif
- Date de création : 2024
- Dernière mise à jour : 26 Décembre 2025

Métriques du projet

- Nombre de modèles : 9 (AppUser, Recipe, Comment, Rating, Stadium, Place, MatchAnnouncement, MatchParticipation, MatchMessage)

- Nombre de services : 8 (AuthService, RecipeService, MatchService, StadiumService, PlaceService, ImageService, PDFService, ThemeService)
- Nombre de providers Riverpod : 40+
- Nombre d'écrans : 10+ (Splash, Auth, Home, Match, Profile, Settings, etc.)
- Dépendances principales : 15+

Chapitre 2

Description du Projet

2.1 Vision Générale

MatchUp Football App est une plateforme mobile conçue pour connecter les passionnés de football et faciliter l'organisation de matchs locaux.

2.2 Public Cible

- Joueurs amateurs et semi-professionnels
- Propriétaires / gestionnaires de stades
- Communautés locales et régionales
- Gamers cherchant une expérience sociale autour du sport

2.3 Proposition de Valeur

1. Facilité d'accès : interface intuitive pour créer et rejoindre des matchs
2. Gestion intelligente : gestion automatisée des participants et positions
3. Communication fluide : messagerie en temps réel
4. Découverte locale : carte des stades disponibles
5. Sécurité : authentification robuste et profils vérifiés

2.4 État Actuel du Projet

L'application a évolué d'une application de recettes vers une plateforme complète de gestion de matchs de football. Le pivot fonctionnel a introduit : annonces, participations, gestion des stades, chat en temps réel, et une architecture scalable.

Chapitre 3

Objectifs et Périmètre

3.1 Objectifs Principaux

Phase 1 – MVP

- Authentification utilisateur (Email/Password)
- Création et gestion des matchs
- Participation et acceptation
- Gestion des stades et emplacements
- UI moderne et responsive

Phase 2 – Améliorations (en cours)

- Messagerie en temps réel
- Notifications push
- Profils enrichis
- Historique des matchs et statistiques
- Intégration des paiements (prévue)

Phase 3 – Futures extensions

- Calendrier de matchs
- Classements / ligues
- Galerie photos
- Streaming en direct
- Communauté / forums

3.2 Périmètre

Inclus : gestion matchs, authentification, profils, Firestore, Storage, multi-plateforme, thème clair/sombre, recherche/filtrage.

Exclus (pour maintenant) : paiement, calendriers externes, streaming vidéo, réseaux sociaux.

3.3 KPIs

- Temps de chargement < 2 secondes
- Rétention utilisateur $> 40\%$
- Disponibilité 99.9%
- 1000+ utilisateurs simultanés
- Satisfaction $> 4.5/5$

Chapitre 4

Architecture Générale

4.1 Architecture Globale



```

    COUCHE DONN ES (Data & Infrastructure)
    Firebase (Firestore/Auth/Storage) | Local Storage/Cache

```

4.2 Patterns Architecturaux

- MVVM + Riverpod
- Repository Pattern (Services comme abstraction d'accès données)
- Provider Pattern (Riverpod) : injection + réactivité
- Singleton Pattern (services globaux)
- Stream Pattern (Firestore streams)

4.3 Flux de Données

```

Utilisateur -> Widget/Screen -> Provider Riverpod -> Service
-> Firebase Firestore -> Données mises à jour -> Provider
    notifie
-> Widget rebuild -> UI mise à jour

```

4.4 Principes de Conception

- SRP (Single Responsibility)
- OCP (Open/Closed)
- Dependency Inversion
- DRY
- SOLID (autant que possible)

Chapitre 5

Technologies Utilisées

5.1 Frontend

Technologie	Version	Utilisation
Flutter	Latest	Framework principal
Dart	Latest	Langage de programmation
Provider	Latest	Gestion d'état (héritage / option)
GetX	Optional	Navigation / état (optionnel)

5.2 Backend & Services

Service	Utilisation
Firebase Firestore	Base NoSQL temps réel
Firebase Authentication	Authentification utilisateur
Firebase Storage	Stockage images/fichiers
Firebase Cloud Functions	Logique serveur serverless

5.3 Dépendances clés (extrait)

```
flutter_riverpod: ^2.5.0
firebase_core: ^3.1.0
firebase_auth: ^5.1.0
cloud_firestore: ^5.0.0
firebase_storage: ^12.0.0
cached_network_image: ^3.3.1
image_picker: ^1.0.7
pdf: ^3.11.0
```

```
printing: ^5.12.0
```

Chapitre 6

Structure du Projet

6.1 Hiérarchie des dossiers

```
flutter_recipes_app/  
  lib/ (Dart)  
  android/ (Kotlin)  
  ios/ (Swift)  
  web/  
  windows/  
  linux/  
  macos/  
  assets/  
  pubspec.yaml  
  README.md
```

6.2 Arborescence lib/src

```
lib/src/  
  models/  
  views/screens/  
  controllers/  
  services/  
  widgets/  
  utils/  
  constants/
```

Chapitre 7

Fonctionnalités Principales

7.1 Gestion des recettes (héritage)

- Consultation : liste, recherche, filtrage
- Ajout / modification / suppression
- Favoris, collections, partage

7.2 Authentification & profil

- Inscription, connexion, reset mot de passe
- Profil : avatar, bio, statistiques, préférences

7.3 Fonctionnalités communautaires

- Commentaires, likes, follows
- Messages privés
- Annonces de match, participations, chat de match

7.4 Notifications

- Notifications temps réel
- Préférences notification
- Centre de notifications

7.5 Gestion des données & sécurité

- Synchronisation cloud (Firebase)

- Mode hors ligne (cache)
- Règles de sécurité Firestore
- RGPD : suppression compte/données

Chapitre 8

Modèles de Données

Exemples de modèles utilisés dans l'application.

8.1 Modèle Recette

```
class Recipe {  
    final String id;  
    final String title;  
    final String description;  
    final String userId;  
    final List<String> ingredients;  
    final List<String> steps;  
    final int preparationTime;  
    final int cookingTime;  
    final int servings;  
    final double rating;  
    final int difficulty;  
    final String category;  
    final List<String> tags;  
    final List<String> imageUrls;  
    final DateTime createdAt;  
    final DateTime updatedAt;  
    final bool isPublic;  
}
```

8.2 Modèle Utilisateur

```
class User {  
    final String uid;
```

```
    final String email;
    final String displayName;
    final String photoUrl;
    final String bio;
    final List<String> favoriteRecipeIds;
    final List<String> createdRecipeIds;
    final List<String> followingIds;
    final List<String> followersIds;
    final Map<String, dynamic> preferences;
    final DateTime createdAt;
    final bool isVerified;
}
```

8.3 Modèle Match

```
class Match {
    final String id;
    final String creatorId;
    final String title;
    final String description;
    final int totalPlayersNeeded;
    final List<String> participantIds;
    final DateTime createdAt;
    final DateTime eventDate;
    final Map<String, dynamic> metadata;
}
```

Chapitre 9

Services et Providers

9.1 Exemple Provider (StateNotifier)

```
final recipeProvider = StateNotifierProvider.autoDispose((ref) {
  return RecipeNotifier();
});

class RecipeNotifier extends StateNotifier<List<Recipe>> {
  RecipeNotifier() : super([]);

  Future<void> fetchRecipes() async {
    // Logique de r cup ration
  }

  void addRecipe(Recipe recipe) {
    state = [...state, recipe];
  }
}
```

Chapitre 10

Intégration Firebase

10.1 Initialisation Firebase

```
void main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  
  await Firebase.initializeApp(  
    options: DefaultFirebaseOptions.currentPlatform,  
  );  
  
  runApp(const RecipesApp());  
}
```

10.2 Exemples de requêtes Firestore

```
// Recettes publiques  
final recipes = await FirebaseFirestore.instance  
  .collection('recipes')  
  .where('isPublic', isEqualTo: true)  
  .orderBy('createdAt', descending: true)  
  .limit(20)  
  .get();
```

10.3 Règles de sécurité Firestore (extrait)

```
rules_version = '2';  
service cloud.firestore {
```

```
match /databases/{database}/documents {  
  match /recipes/{document=**} {  
    allow read: if resource.data.isPublic == true;  
    allow create: if request.auth != null;  
    allow update, delete: if request.auth.uid == resource.data.  
      userId;  
  }  
}  
}
```

Chapitre 11

Écrans et Navigation

L'application contient 10+ écrans (Splash, Auth, Home, Match, Profile, Settings, etc.) et une navigation structurée (routes).

Chapitre 12

Configuration et Installation

12.1 Prérequis

```
flutter --version  
dart --version
```

12.2 Installation

```
git clone <repository_url>  
cd flutter_recipes_app  
flutter pub get  
flutter run
```

12.3 Firebase (résumé)

- Ajouter Android / iOS / Web dans Firebase Console
- Placer `google-services.json` et `GoogleService-Info.plist`
- Générer `firebase_options.dart` via `flutterfire configure`

Chapitre 13

Détails Techniques

13.1 Sérialisation JSON (json_serializable)

```
@JsonSerializable()
class Recipe {
  @JsonKey(name: 'recipe_id')
  final String id;
  final String title;

  Recipe({required this.id, required this.title});

  factory Recipe.fromJson(Map<String, dynamic> json) =>
    _$RecipeFromJson(json);

  Map<String, dynamic> toJson() => _$RecipeToJson(this);
}
```

Chapitre 14

Gestion de l'État (Riverpod)

Riverpod fournit :

- Injection de dépendances
- Caching et scopes
- Réactivité aux streams Firestore
- Meilleure testabilité des composants

Chapitre 15

Plateformes Supportées

15.1 Mobile

- Android : min API 21, cible API 33+
- iOS : min iOS 11, cible iOS 15+

15.2 Desktop

- Windows : Windows 10+
- macOS : 10.11+
- Linux : Ubuntu 20.04+ / Fedora 32+ / Debian 10+

15.3 Web

Chrome 90+, Firefox 88+, Safari 14+, Edge 90+.

Chapitre 16

Gestion des Dépendances

16.1 Extrait pubspec.yaml

```
dependencies:  
  flutter:  
    sdk: flutter  
  flutter_riverpod: ^2.5.0  
  firebase_core: ^3.1.0  
  firebase_auth: ^5.1.0  
  cloud_firestore: ^5.0.0  
  firebase_storage: ^12.0.0
```

16.2 Commandes utiles

```
flutter pub get  
flutter pub upgrade  
flutter pub outdated
```

Chapitre 17

Tests et Validation

- Tests unitaires (models)
- Tests widget (UI)
- Tests d'intégration
- Analyse statique (flutter analyze)
- Formatage (dart format)

Chapitre 18

Défis et Solutions

18.1 Synchronisation en temps réel

Approche basée sur Streams + Riverpod, pagination et cache local.

18.2 Sécurité

Règles Firestore granulaires et validation des accès.

18.3 Performances

Lazy loading, caching images, virtualisation des listes.

18.4 Multi-plateforme

Widgets standard, détection de plateforme, tests sur émulateurs et appareils.

Chapitre 19

Perspectives Futures

19.1 Court terme (1–3 mois)

- Notifications push (FCM)
- Recherche avancée
- Mode hors ligne amélioré
- Localisation multi-langues

19.2 Moyen terme (3–6 mois)

- Recommandations IA
- Chat temps réel amélioré
- Analytics (Firebase Analytics)
- Publication sur stores

19.3 Long terme (6+ mois)

- Fonctionnalités premium / paiement
- API publique
- ML pour suggestions personnalisées
- Communautés et événements en ligne

Chapitre 20

Conclusion

MatchUp Football App est un projet ambitieux démontrant l'utilisation de Flutter et Firebase pour construire une application multi-plateforme, réactive et évolutive. L'architecture (MVVM + Riverpod) et la séparation en couches facilitent la maintenance, les tests et l'évolution future.

Annexes

Ressources utiles

- Documentation Flutter : <https://flutter.dev/docs>
- Firebase pour Flutter : <https://firebase.flutter.dev>
- Dart : <https://dart.dev/guides>
- Material Design 3 : <https://m3.material.io/>

Commandes utiles

```
flutter clean
flutter pub get
flutter pub run build_runner build --delete-conflicting-outputs
flutter test
flutter analyze
dart format lib/ -l 80
```